

Home Assistant Schichtplaner

Bilinguale Masterdokumentation V7.0

Erstellt: 23.05.2026 19:50

1. Projektübersicht

■ ■
Dieses Dokument kombiniert die ursprüngliche Gemini-Entwicklerdokumentation, die erweiterten Architekturanalysen, die Dashboard-Evolution, die Statistikentwicklung sowie die GitHub-Release-Vorbereitung zu einer einzigen vollständigen Masterdokumentation.

■ ■
This document combines the original Gemini developer documentation, extended architecture analysis, dashboard evolution, statistics development, and GitHub release preparation into one complete master documentation.

2. Projektziel

■ ■
Das Ziel des Projekts war ursprünglich ein einfacher Schichtkalender. Im Verlauf entwickelte sich das System jedoch zu einer vollständigen Home-Assistant-Planungsplattform.

■ ■
The original goal of the project was a simple shift calendar. Over time, however, the system evolved into a complete Home Assistant planning platform.

■ ■
Finale Ziele: - Mehrpersonenplanung - Mobile Bedienung - Familienkalender - Statistiksystem - Mehrtagealogik - Fernzugriff - Historische Datenerhaltung

■ ■
Final goals: - Multi-user planning - Mobile usability - Family calendar - Statistics system - Multi-day logic - Remote access - Historical data preservation

3. Systemarchitektur

■ ■
Die Architektur wurde schrittweise in mehrere logisch getrennte Ebenen aufgeteilt.

■ ■
The architecture was gradually separated into multiple logical layers.

Layer	Beschreibung / Description
Frontend	Lovelace UI, Touch Dashboard
Logic Layer	Jinja2 Templates, Routing, Scripts
Storage Layer	Google Calendar / Local Calendar
Statistics Layer	calendar.get_events + input_text
Network Layer	WireGuard VPN + FritzBox

4. Dashboard Evolution



Die Dashboardentwicklung war einer der wichtigsten Teile des Projekts. Die Benutzeroberfläche entwickelte sich von einfachen Entity-Karten zu einer vollständigen mobilen Touch-Oberfläche.



Dashboard development became one of the most important parts of the project. The interface evolved from simple entity cards into a complete mobile touch interface.



Wichtige Entwicklungen: - große Touchflächen - Mehrfach-Datumswahl - Personenrouting - dynamische Schichtkarten - mobile Optimierung



Important improvements: - large touch areas - multi-date selection - person routing - dynamic shift cards - mobile optimization

5. Geisterbuchungsproblem



Eines der größten Probleme war das sogenannte Geisterbuchungsproblem. Versteckte Dropdownwerte konnten sichtbare Benutzerauswahlen überschreiben.



One of the biggest issues was the so-called ghost booking problem. Hidden dropdown values could overwrite visible user selections.

```
v_schicht: >-
{% if v_person == 'Sascha' %}
  {{ states('input_select.planung_schichten_sascha') }}
{% elif v_person == 'Sabine' %}
  {{ states('input_select.planung_schichten_sabine') }}
{% else %}
  {{ states('input_select.schicht_auswahl') }}
{% endif %}
```



Dieser Fix eliminierte die falschen Hintergrundzustände.



This fix eliminated invalid hidden background states.

6. Statistiksystem



Die Statistik entwickelte sich zum komplexesten Bereich des gesamten Projekts.



The statistics system became the most complex part of the entire project.



Die ursprüngliche textbasierte Logik war instabil. Emojis, Schreibweisen und Kalenderänderungen führten zu falschen Ergebnissen.



The original text-based logic was unstable. Emojis, naming differences and calendar modifications caused

incorrect results.

```
ALT:
selectattr('summary', 'search', 'Früh')

NEU:
selectattr('start', 'search', 'T06:00')
```



Die Umstellung auf Zeitlogik verbesserte Stabilität und Reproduzierbarkeit massiv.



Switching to time-based logic massively improved stability and reproducibility.

7. Reload- & Timing-Probleme



Home Assistant behandelte Reloads teilweise wie echte Zustandsänderungen. Dadurch entstanden doppelte Einträge und inkonsistente Statistikwerte.



Home Assistant sometimes treated reloads like real state changes. This caused duplicate entries and inconsistent statistics.

```
condition:
- condition: template
  value_template: >
    {{ trigger.from_state is not none and
      trigger.to_state.state != trigger.from_state.state }}
```



Zusätzlich mussten künstliche Delays eingebaut werden, um Kalender-Cache-Probleme zu vermeiden.



Additional artificial delays were required to avoid calendar cache issues.

```
delay: "00:00:02"
```

8. Kalenderarchitektur



Das System begann mit separaten Kalendern. Später wurde zusätzlich der Familienplaner eingeführt.



The system originally used separate calendars. Later, a shared family planner was introduced.

```
calendar.sascha
calendar.sabine
calendar.familienplaner
```



Dadurch entstand eine Mischarchitektur aus historischen und neuen Datenquellen.



This resulted in a hybrid architecture containing historical and new data sources.

9. GitHub & Open Source Vorbereitung



Das Projekt wurde schrittweise für eine professionelle GitHub-Veröffentlichung vorbereitet.



The project was gradually prepared for a professional GitHub release.

- /helpers
- /scripts
- /automations
- /dashboards
- /docs
- /troubleshooting
- /changelog



Zusätzlich wurden folgende Bereiche vorbereitet: - Semantic Versioning - CHANGELOG - Troubleshooting - Datenschutz - YAML Strukturierung - Screenshot-Dokumentation



Additionally, the following areas were prepared: - Semantic Versioning - CHANGELOG - Troubleshooting - Privacy considerations - YAML structuring - Screenshot documentation

10. Sicherheitsarchitektur



Da das System Familien- und Kalenderdaten verarbeitet, wurden zusätzliche Sicherheitsmaßnahmen umgesetzt.



Since the system processes family and calendar data, additional security measures were implemented.



Sicherheitsmaßnahmen: - WireGuard VPN - keine offenen Ports - lokale Verarbeitung - Backups vor Updates - FritzBox Direktanbindung



Security measures: - WireGuard VPN - no open ports - local processing - backups before updates - direct FritzBox connection

11. Lessons Learned



Die Entwicklung zeigte, dass UI, Kalenderlogik und Backend stärker miteinander verbunden sind als ursprünglich erwartet.



Development showed that UI, calendar logic and backend systems are much more interconnected than originally expected.



Wichtigste Erkenntnisse: - Zeitlogik ist stabiler als Textlogik - Reloads erzeugen Seiteneffekte - mobile Bedienung beeinflusst Backendentscheidungen - Mischsysteme benötigen klare Datenquellen - Versionierung ist extrem wichtig



Key lessons learned: - Time-based logic is more stable than text-based logic - Reloads create side effects - Mobile usability affects backend architecture - Hybrid systems require clear data sources - Versioning is extremely important

12. Finaler Projektstatus



Das Projekt entwickelte sich von einem kleinen privaten Kalender zu einer vollständigen Home-Assistent-Managementplattform.



The project evolved from a small private calendar into a complete Home Assistant management platform.



Finale Features: - Mehrpersonenplanung - Touch-Dashboard - Familienkalender - Statistiksysteem - Mehrtage-logik - historische Datenerhaltung - Reloadschutz - Doppeltermenschutz - Google/Nextcloud Integration



Final features: - Multi-user planning - Touch dashboard - Family calendar - Statistics system - Multi-day logic - historical data preservation - reload protection - duplicate protection - Google/Nextcloud integration

Quellenbasis: Eigene Entwicklungsdokumentation + Gemini Masterdokumentation 2026